

705.641.81 Natural Language Processing: Self-Supervised Models

Homework 1: Background Review + Building a Classifier

Brian Onieal

September 13, 2026

Name: Brian Onieal

Sources used: Course notes on linear algebra; PyTorch documentation (`nn.Linear`, `torch.argmax`, `torch.utils.data`); Gensim `KeyedVectors` documentation; Hugging Face `datasets` documentation.

1 Refreshing the Rows and Columns: Linear Algebra Review

1.1 Basic Operations

With $\alpha = 2$, $x = [0, 1, 2]^\top$, $y = [3, 4, 5]^\top$, $z = [1, 2, -1]^\top$, and

$$A = \begin{bmatrix} 3 & 2 & 2 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \end{bmatrix}.$$

1. $\sum_{i=1}^n x_i y_i = (0)(3) + (1)(4) + (2)(5) = 14$

2. $\sum_{i=1}^n x_i z_i = (0)(1) + (1)(2) + (2)(-1) = 0$

3. $\alpha(x + y) = 2 \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix} = \begin{bmatrix} 6 \\ 10 \\ 14 \end{bmatrix}$

4. $\|x\| = \sqrt{0^2 + 1^2 + 2^2} = \sqrt{5} \approx 2.236$

5. $x^\top = [0 \quad 1 \quad 2]$, a 1×3 row vector.

6. $Ax = \begin{bmatrix} 6 \\ 5 \\ 7 \end{bmatrix}$

$$7. x^\top Ax = \begin{bmatrix} 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 6 \\ 5 \\ 7 \end{bmatrix} = 19$$

1.2 Matrix Algebra Rules

1. $x^\top y = \sum_{i=1}^n x_i y_i$. **True.** This is the definition of the inner product.
2. $x^\top x = \|x\|^2$. **True.** $\sum_i x_i^2$ is the squared Euclidean norm.
3. $x^\top x = xx^\top$. **False.** The left side is a 1×1 scalar; the right side is an $n \times n$ outer product matrix of rank 1. They are not even the same shape.
4. $(x-y)^\top (y-x) = \|x\|^2 - 2x^\top y + \|y\|^2$. **False.** Note that $(x-y)^\top (y-x) = -(x-y)^\top (x-y) = -\|x-y\|^2$, so the left side is the *negative* of the right side. The identity holds only for $(x-y)^\top (x-y)$.
5. $AB = BA$. **False.** Matrix multiplication is not commutative in general. It holds only in special cases, such as when one factor is I or a scalar multiple of I , or when A and B are simultaneously diagonalizable.
6. $A(B+C) = AB+AC$. **True.** Matrix multiplication distributes over addition.
7. $(AB)^\top = A^\top B^\top$. **False.** The correct identity is $(AB)^\top = B^\top A^\top$: transposition reverses the order of the factors.
8. $x^\top Ay = y^\top A^\top x$. **True.** Both sides are 1×1 , and a scalar equals its own transpose, so $x^\top Ay = (x^\top Ay)^\top = y^\top A^\top x$.
9. $A^\top A = I$ if the columns of A are orthonormal. **True.** Entry (i, j) of $A^\top A$ is the inner product of columns i and j , which is 1 when $i = j$ and 0 otherwise.

1.3 Matrix Operations

Let

$$B = \begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}.$$

Is B invertible?

No. Expanding along the first row,

$$\det(B) = 1(2 \cdot 1 - (-1)(-1)) - (-1)((-1)(1) - (-1)(0)) + 0 = 1(1) + 1(-1) = 0.$$

Since $\det(B) = 0$, the matrix is singular and B^{-1} does not exist. The rank is 2. The reason is visible in the structure: every row sums to zero, so $B[1, 1, 1]^\top = 0$ and the vector $[1, 1, 1]^\top$ lies in the null space. (B is the graph Laplacian of a three-node path, and every Laplacian is singular for this reason.)

Is B diagonalizable?

Yes. B is real and symmetric, so the spectral theorem guarantees it is diagonalizable, and in fact orthogonally diagonalizable. Independently, the characteristic polynomial factors as

$$\det(B - \lambda I) = -\lambda(\lambda - 1)(\lambda - 3),$$

giving three distinct eigenvalues $\lambda_1 = 0$, $\lambda_2 = 1$, $\lambda_3 = 3$. Distinct eigenvalues alone are sufficient for diagonalizability.

The corresponding eigenvectors are

$$v_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}, \quad v_3 = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix},$$

so that $B = PDP^{-1}$ with

$$P = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & -2 \\ 1 & -1 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{bmatrix}, \quad P^{-1} = \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/2 & 0 & -1/2 \\ 1/6 & -1/3 & 1/6 \end{bmatrix}.$$

Because the eigenvectors are mutually orthogonal, normalizing them gives the orthogonal diagonalization $B = QDQ^\top$ with

$$Q = \begin{bmatrix} 1/\sqrt{3} & 1/\sqrt{2} & 1/\sqrt{6} \\ 1/\sqrt{3} & 0 & -2/\sqrt{6} \\ 1/\sqrt{3} & -1/\sqrt{2} & 1/\sqrt{6} \end{bmatrix}, \quad Q^\top Q = I.$$

2 Taking Chances: Probability Review

2.1 Basic Probability

1. Of the 36 equally likely ordered outcomes, the pairs with $|a - b| \geq 3$ number 12: six with difference 3, four with difference 4, and two with difference 5. So the win probability is $12/36 = 1/3$ and

$$\mathbb{E}[\text{payout}] = 15 \cdot \frac{1}{3} = \boxed{\$5}.$$

A fair ticket price is \$5.

2. Mutually exclusive, so $P(A \cup B) = P(A) + P(B)$:

$$0.95 = 0.40 + P(B) \implies P(B) = 0.55.$$

3. Independent, so $P(A, B) = P(A)P(B)$ and

$$0.95 = 0.40 + P(B) - 0.40 P(B) = 0.40 + 0.60 P(B) \implies P(B) = \frac{0.55}{0.60} = \frac{11}{12} \approx 0.9167.$$

2.2 Expectations and Variance

The coin is re-drawn on every trial, so each of the three trials is an independent Bernoulli draw with the same marginal success probability:

$$P(H) = \frac{1}{2}(0.3) + \frac{1}{2}(0.9) = 0.6.$$

Therefore $X \sim \text{Binomial}(n = 3, p = 0.6)$.

1. $\mathbb{E}[X] = np = 3(0.6) = 1.8$
2. $\text{Var}[X] = np(1 - p) = 3(0.6)(0.4) = 0.72$
3. The pmf of X is

$$P(X=0) = 0.064, \quad P(X=1) = 0.288, \quad P(X=2) = 0.432, \quad P(X=3) = 0.216,$$

and since $Y = \frac{1}{2+X}$ is a nonlinear function of X , we must average over the pmf rather than substitute $\mathbb{E}[X]$:

$$\mathbb{E}[Y] = \frac{0.064}{2} + \frac{0.288}{3} + \frac{0.432}{4} + \frac{0.216}{5} = 0.032 + 0.096 + 0.108 + 0.0432 = \frac{349}{1250} = 0.2792.$$

For contrast, $\frac{1}{2+\mathbb{E}[X]} = \frac{1}{3.8} \approx 0.2632$, which is not the same number. Jensen's inequality predicts this: $1/(2+x)$ is convex, so $\mathbb{E}[Y] \geq 1/(2 + \mathbb{E}[X])$.

2.3 A Variance Paradox?

There is no contradiction. The two expressions describe different random variables.

The result $\text{Var}[X_1 + \cdots + X_n] = n\sigma^2$ requires the summands to be uncorrelated. In general,

$$\text{Var}\left[\sum_i X_i\right] = \sum_i \text{Var}[X_i] + \sum_{i \neq j} \text{Cov}(X_i, X_j),$$

and independence kills every cross term, leaving $n\sigma^2$.

In $X + X$ the two summands are the same random variable, not two independent draws. A variable is perfectly correlated with itself, so $\text{Cov}(X, X) = \text{Var}[X] = \sigma^2$ and the cross terms survive:

$$\text{Var}[X + X] = \sigma^2 + \sigma^2 + 2\sigma^2 = 4\sigma^2 = \text{Var}[2X].$$

The confusion comes from reading “identically distributed” as “identical.” X_1 and X_2 being i.i.d. means they follow the same distribution F but realize independently: on any given draw they take different values. X and X realize the same value every time. Adding two independent draws lets high and low realizations partially cancel, which is why the variance grows like n . Doubling a single draw cannot cancel anything, so the deviation doubles and the variance quadruples.

3 Calculus Review

3.1 One-Variable Derivatives

1. $f(x) = 3x^2 - 2x + 5 \implies f'(x) = 6x - 2$
2. $f(x) = x(1 - x) = x - x^2 \implies f'(x) = 1 - 2x$
3. With $p(x) = \frac{1}{1 + e^{-x}}$ and $f(x) = x - \log p(x)$:

$$\frac{d}{dx} \log p(x) = \frac{p'(x)}{p(x)} = \frac{p(x)(1 - p(x))}{p(x)} = 1 - p(x),$$

so

$$f'(x) = 1 - (1 - p(x)) = p(x).$$

Equivalently, $\frac{d}{dx} \log p(x) = p(-x)$ and $f'(x) = 1 - p(-x)$, which is the same thing by the identity $p(x) = 1 - p(-x)$ given in the hint. So $\boxed{f'(x) = p(x)}$.

3.2 Multi-Variable Derivatives

1. $f(x) = x_1^2 + e^{x_2}$:

$$\nabla f(x) = \begin{bmatrix} 2x_1 \\ e^{x_2} \end{bmatrix}$$

2. $f(x) = e^{x_1 + x_2 x_3}$:

$$\nabla f(x) = e^{x_1 + x_2 x_3} \begin{bmatrix} 1 \\ x_3 \\ x_2 \end{bmatrix}$$

3. $f(x) = a^\top x = \sum_i a_i x_i$:

$$\nabla f(x) = a$$

4. $f(x) = x^\top A x$ with $A = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}$. In general $\nabla f = (A + A^\top)x$, and A is symmetric here, so $\nabla f = 2Ax$:

$$\nabla f(x) = \begin{bmatrix} 4x_1 - 2x_2 \\ -2x_1 + 2x_2 \end{bmatrix}$$

Checking by expansion, $f(x) = 2x_1^2 - 2x_1x_2 + x_2^2$, which gives the same partials.

5. $f(x) = \frac{1}{2}\|x\|^2 = \frac{1}{2}\sum_{i=1}^d x_i^2$:

$$\nabla f(x) = x$$

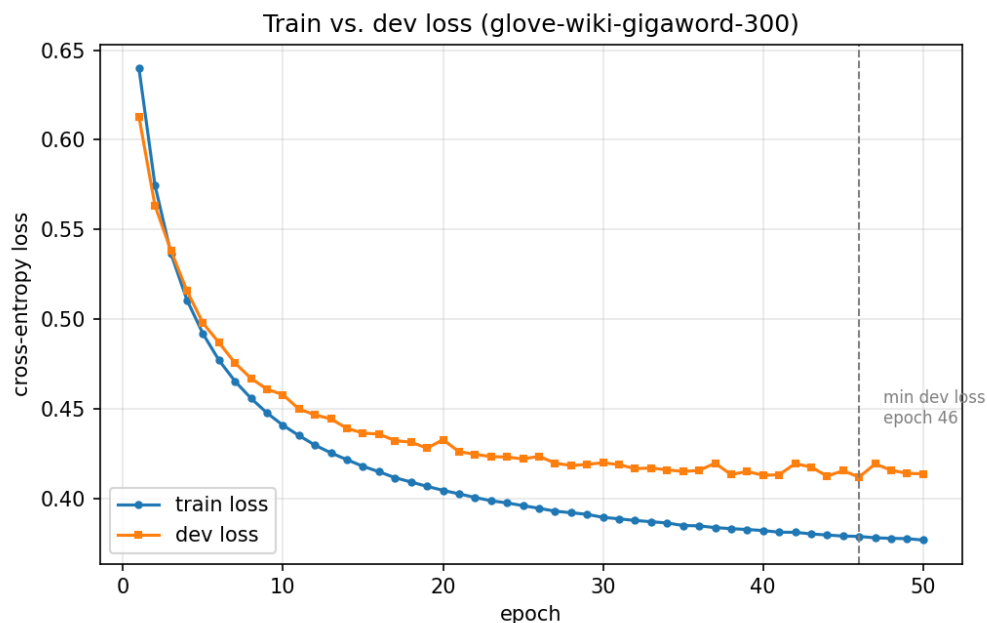
4 Algorithms and Data Structures Review

1. Merge sort on n numbers: $O(n \log n)$. The recursion splits to depth $\log n$ and each level does $O(n)$ merging work.
2. Third-largest element of an unsorted list of n numbers: $O(n)$. One pass while maintaining the top three seen so far. Sorting first would be $O(n \log n)$ and is unnecessary because the rank is a fixed constant.
3. Smallest element greater than 0 in a sorted list of n numbers: $O(\log n)$ by binary search for the boundary.
4. Value associated with a key in a hash table of n entries: $O(1)$ average case, assuming a good hash function and a bounded load factor. Worst case degrades to $O(n)$ if every key collides into one bucket.
5. Matrix-vector product Ax with A of size $n \times d$ and x of size $d \times 1$: $O(nd)$. Each of the n output entries costs d multiply-adds.
6. Quadratic form $x^\top Ax$ with A of size $d \times d$: $O(d^2)$. Compute Ax in $O(d^2)$, then the inner product with $x^\top$ in $O(d)$, and $O(d^2) + O(d) = O(d^2)$.
7. Matrix multiplication AB with A of size $m \times n$ and B of size $n \times d$: $O(mnd)$ using the standard triple-loop algorithm. (Subcubic methods such as Strassen exist but are not the assumed baseline.)

5 Programming

5.1 5.2.7 Run the pipeline: Train Loss vs. Dev Loss

Configuration: glove-wiki-gigaword-300, 20k train / 1k dev, single `nn.Linear(300, 2)`, Adam at 10^{-3} , batch size 64, 50 epochs, seed 42.

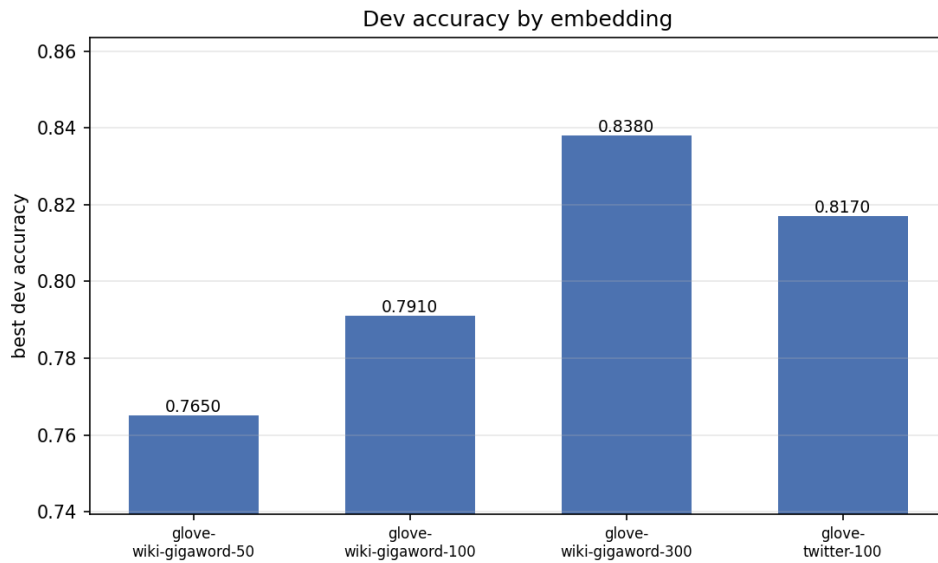
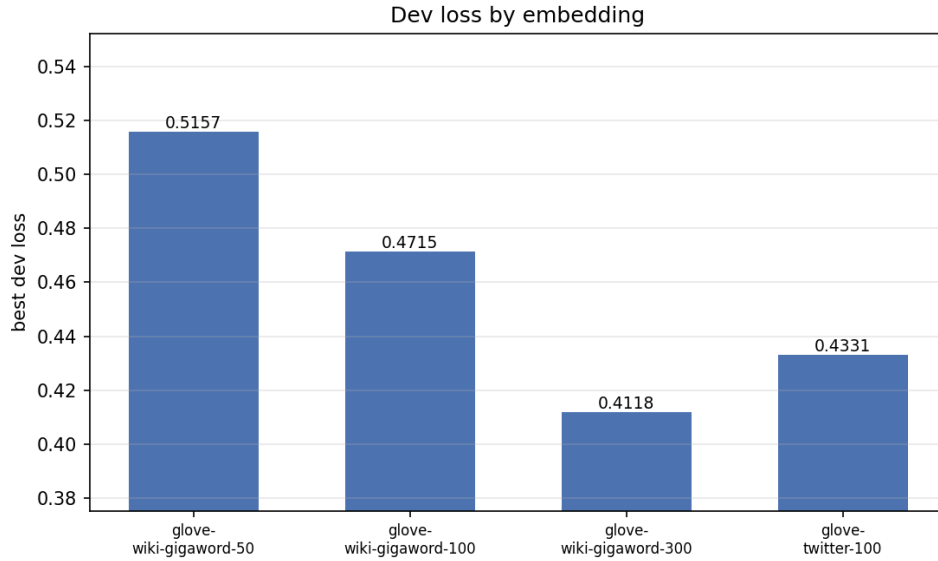


Answer:

Both curves fall steeply for the first ten epochs and then separate: train loss decreases monotonically to 0.3767 by epoch 50, while dev loss flattens near 0.415 after roughly epoch 30 and bottoms out at 0.4118 on epoch 46. The gap between them opens from -0.027 at epoch 1 to $+0.037$ at epoch 50, which indicates the model has started fitting the training set specifically rather than learning signal that transfers. The divergence is mild instead of the classic U-shaped overfitting curve, and the reason is capacity: a 300-dimensional average feeding one linear layer gives the model only 602 parameters against 20,000 training examples, so it has no room to memorize and dev loss plateaus rather than turning back up.

5.2 5.2.8 Run the pipeline: Explore Different Word Embeddings

The four configurations are chosen as a two-factor comparison. The first three hold the training corpus fixed (Wikipedia + Gigaword) and vary only dimension, isolating embedding size. The fourth holds dimension fixed at 100 against glove-wiki-gigaword-100 and swaps the corpus to Twitter, isolating training domain.



Embedding	Dim	Best dev loss	Best dev acc
glove-wiki-gigaword-50	50	0.5157	0.7650
glove-wiki-gigaword-100	100	0.4715	0.7910
glove-wiki-gigaword-300	300	0.4118	0.8380
glove-twitter-100	100	0.4331	0.8170

Answer:

Holding the corpus fixed, accuracy rises monotonically with dimension (0.7650, 0.7910, 0.8380 at 50, 100, and 300 dimensions), which is expected because the classifier is a single linear layer with no capacity to recover semantic detail the representation has already discarded. The more informative comparison is **glove-twitter-100** against **glove-wiki-gigaword-100** at matched dimension, where Twitter wins 0.8170 to 0.7910. That is not a vocabulary-coverage effect: measuring both

vocabularies against the dev set, the Twitter embedding actually covers less of it (96.45% of tokens and 85.14% of types, versus 97.43% and 89.79% for Wikipedia + Gigaword) despite a vocabulary three times larger, so the advantage has to come from register, since Twitter text is informal and opinion-bearing and therefore much closer to movie-review language than encyclopedia and newswire prose.

One ceiling is worth naming: because **featurize** averages every token vector in a review, word order and negation are discarded before the classifier ever sees the input, so “not good” and “good” produce nearly identical features. That caps what any embedding choice can buy, and it is a property of the representation rather than of the embeddings.

A Completed Skeleton Code (TODO Sections)

The five TODO blocks are reproduced below. The full completed code base is submitted separately as a .zip.

featurize in model.py

```
def featurize(sentence, embeddings):
    """Convert a raw review string into a single averaged embedding tensor
    """
    tokens = sentence.lower().split()

    vectors = []
    for token in tokens:
        if token in embeddings:          # skip out-of-vocabulary tokens
            vectors.append(embeddings[token])

    if len(vectors) == 0:                # guard: review with no known
        tokens                           tokens
        return torch.zeros(embeddings.vector_size)

    stacked = np.stack(vectors, axis=0)   # [num_tokens, embed_size]
    averaged = np.mean(stacked, axis=0)   # [embed_size]

    return torch.from_numpy(averaged).float()
```

create_tensor_dataset in model.py

```
def create_tensor_dataset(raw_data, embeddings):
    all_features, all_labels = [], []
    for label, text in raw_data:
        all_features.append(featurize(text, embeddings))
        all_labels.append(label)

    features_tensor = torch.stack(all_features)          # [N, embed_size]
    labels_tensor = torch.tensor(all_labels, dtype=torch.long) # [N]

    return TensorDataset(features_tensor, labels_tensor)
```

SentimentClassifier.__init__ and forward in model.py

```
class SentimentClassifier(nn.Module):
    def __init__(self, embed_size, num_classes=2):
        super().__init__()
        self.layer_pred = nn.Linear(embed_size, num_classes)
        self.loss_fn = nn.CrossEntropyLoss()

    def forward(self, inp):
```

```
# inp: [batch_size, embed_size] -> logits: [batch_size,
      num_classes]
return self.layer_pred(inp)
```

accuracy in model.py

```
def accuracy(logits, labels):
    """Return a tensor of 0s and 1s marking each prediction correct or not
    ."""
    predictions = torch.argmax(logits, dim=1) # [batch_size]
    return (predictions == labels).float()
```